



CS-104L Object Oriented Programming Project REPORT

SECTION: "D"

SUBMITTED TO:

Mam Laiba

SUBMITTED BY:

Shahzeb (25-CS-099)

Abdullah Shahid(25-CS-069)

DATE: 20-06-2026

**DEPARTMENT OF COMPUTER SCIENCE
HITEC UNIVERSITY TAXILA**

Library Management System

CS-104 Object Oriented Programming Project | HITEC University, Taxila

1. Introduction

This document explains the design decisions, OOP concepts, and program logic behind the Library Management System. The application is a single-file C++ console program that lets a librarian add books and magazines to a catalogue, register members, issue and return items, search the catalogue, and persist everything to disk between runs. It was built specifically to demonstrate classes and objects, encapsulation, inheritance, polymorphism, abstraction, static members, const correctness, a friend function, file handling, and exception handling, all of which are required by the project specification.

2. Overall Structure

The program is organized into three layers. The first layer is a small hierarchy of custom exception classes used for error handling. The second layer is the data model: two abstract classes, `LibraryItem` and `Person`, each extended by two concrete classes. The third layer is the `Library` class, which owns all the data and exposes the operations the rest of the program is allowed to perform on it. `main()` only talks to the `Library` object through a text menu; it never touches the catalogue or member list directly.

Layer	Classes	Responsibility
Errors	<code>LibraryException</code> and 6 subclasses	Represent everything that can go wrong
Data model	<code>LibraryItem</code> , <code>Book</code> , <code>Magazine</code> , <code>Person</code> , <code>Member</code> , <code>Librarian</code>	Represent catalogue items and people
Controller	<code>Library</code>	Owns the data, does all file I/O, enforces the rules
Interface	<code>main()</code> , <code>showMenu()</code> , <code>getValidatedChoice()</code> , <code>getLine()</code>	Talks to the user, drives the loop

Table 1: The four layers of the program.

3. OOP Concepts Used and Where

Classes and Objects

Every entity in the program is modelled as a class: `Book`, `Magazine`, `Member`, `Librarian` and `Library`. `main()` creates real objects from them, for example a new `Book` object is created every time the user chooses 'Add Book' from the menu.

Encapsulation

Data fields are kept private or protected and can only be reached through public methods. The clearest example is the `Library` class: its items and members vectors are private, so the menu code in `main()` cannot add or delete a book directly, it has to call `library.addBook(...)` or `library.removeItem(...)`, which means every change automatically goes through validation and gets logged.

Inheritance

Book and Magazine both inherit from LibraryItem, and Member and Librarian both inherit from Person. This means the shared fields (id, title, issued for items; id, name for people) and shared behaviour only have to be written once, in the base class, instead of being copy-pasted into every derived class.

Polymorphism

LibraryItem::displayDetails() and Person::displayInfo() are declared virtual and overridden differently in each derived class. Library::displayAllItems() simply loops over a vector<LibraryItem*> and calls item->displayDetails() on each one, the correct version, Book's or Magazine's, runs automatically depending on what the pointer actually points to at run time.

Abstraction

LibraryItem and Person are abstract classes: they declare pure virtual functions (displayDetails() = 0, getType() = 0, etc.) and can never be instantiated on their own. They only exist to describe what every item or person must be able to do, the concrete classes fill in how.

Friend Function

operator<<(ostream&, const Book&) is declared as a friend inside the Book class. It is not a member function, but it is still allowed to read Book's private fields (author, isbn) directly so it can print a short summary like Book{B001, The Hobbit, J.R.R. Tolkien} without needing public getters for that one purpose.

Static Members

LibraryItem::totalItems and Member::memberCount are static, meaning there is exactly one copy shared by every object of that class, not one copy per object. They are incremented in the constructor and decremented in the destructor (for totalItems), and exposed through static getter functions so the menu option 'View System Statistics' can report them without needing a specific object.

Const Correctness

Almost every getter (getID(), getTitle(), displayDetails(), canBorrow(), etc.) is marked const, which is a promise to the compiler that the function will not modify the object. Member::MAX_BORROW_LIMIT is a static const int, a single constant value shared by the whole class that can never be changed once compiled.

4. Class-by-Class Explanation

LibraryException (and 6 subclasses)

A small inheritance tree: FileException, InvalidInputException, ItemNotFoundException, MemberNotFoundException, BorrowLimitException and DuplicateEntryException all inherit from LibraryException, which itself inherits from std::exception. Each subclass just prefixes the message with a category label, e.g. "File Error: ", so the same catch (const LibraryException& e) block in main() can catch any of them and e.what() already explains exactly what went wrong.

LibraryItem (abstract base)

Holds the three fields every catalogue item needs: itemID, title, issued. Declares displayDetails(), getType() and serialize() as pure virtual so every derived class is forced to provide its own version. Also owns the static totalItems counter.

Book

Adds author, isbn and genre on top of LibraryItem. serialize() turns the object into one CSV line starting with the word "Book" so loadData() knows how to rebuild it later. Also declares the friend operator<< mentioned above.

Magazine

Adds publisher and issueNumber instead. Having a second LibraryItem subclass is what makes the polymorphism in displayAllItems() and searchItem() meaningful, the same loop has to work correctly for two different real types.

Person (abstract base)

Mirrors LibraryItem but for people: holds id and name, declares displayInfo() and getRole() as pure virtual.

Member

Adds a vector<string> of borrowed item IDs and enforces MAX_BORROW_LIMIT (3) through canBorrow() and addBorrowedItem(), which throws BorrowLimitException once the member is already at the limit. serializeBorrowed() joins the borrowed IDs with semicolons (e.g. "B001;M002") for saving to members.csv.

Librarian

A second, simpler Person subclass. It exists mainly to show that Person can be extended in more than one way, the same way Book and Magazine both extend LibraryItem.

Library (the controller)

Owns the vector<LibraryItem*> catalogue and the vector<Member> member list privately. Every public method first validates its input, throwing one of the custom exceptions if something is wrong, then performs the change and calls logTransaction() to record it. The destructor deletes every dynamically allocated LibraryItem so there are no memory leaks when the program ends.

5. File Handling Logic

The program reads and writes three plain text files in the same folder as the executable:

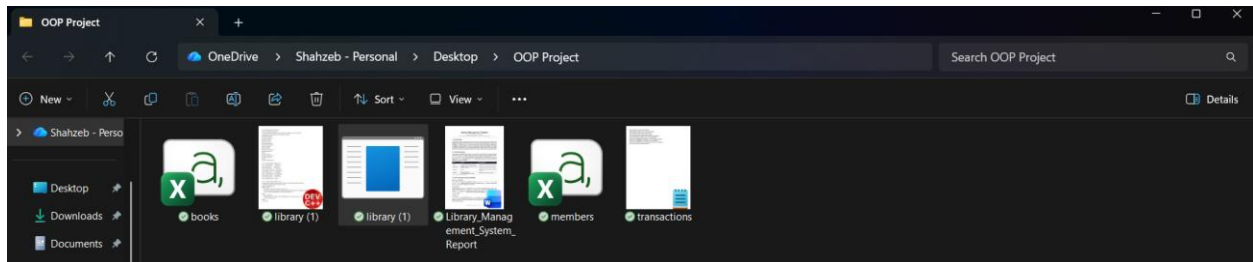
- books.csv - one line per catalogue item, e.g. Book,B001,The Hobbit,J.R.R. Tolkien,9780261103283,Fantasy,0. The first field tells loadData() whether to rebuild a Book or a Magazine.
- members.csv - one line per member, e.g. MEM1,Ali Khan,B001;M002, where the last field is the semicolon-separated list of currently borrowed item IDs.
- transactions.txt - a plain append-only log. Every successful add, issue, return, removal or registration calls logTransaction(), which opens the file in std::ios::app mode and adds one line, so the file keeps growing across sessions instead of being overwritten.

Library's constructor calls loadData() automatically, and saveData() runs whenever the user picks 'Save Data Now' or 'Save & Exit' from the menu. Both books.csv and members.csv use ifstream/ofstream, and each line is split on commas using a stringstream.

Two things make the file handling resilient rather than fragile. First, if a file simply does not exist yet (first run), is_open() returns false and the program just prints a notice and starts with empty data instead of crashing. Second, while loading, every line is parsed inside its own try/catch: if a line is missing fields, has an unrecognized type label, or has a non-numeric issue number for a magazine, a FileException is thrown, caught

immediately, printed as a warning, and the loop simply moves on to the next line rather than aborting the whole load.

Image of File Creation While Running The program First Time:



Created 2 Excel files for storing the list of Books and members of Library. And one text (.txt) file for storing the history of transaction.

6. Exception Handling Logic

Every Library method validates its own inputs and throws a specific exception type rather than returning an error code, which keeps the menu code in main() simple, it just tries the operation and reports whatever comes back.

Exception	Thrown when...
InvalidInputException	An ID/title/name field is empty, a menu choice is out of range, or text was typed where a number was expected
DuplicateEntryException	addBook(), addMagazine() or registerMember() is called with an ID that already exists
ItemNotFoundException	issueItem(), returnItem() or removeItem() is given an item ID that is not in the catalogue
MemberNotFoundException	issueItem() or returnItem() is given a member ID that was never registered
BorrowLimitException	addBorrowedItem() is called for a member who already has 3 items out
FileException	A CSV file cannot be opened for writing, or a line being loaded is corrupted/unrecognized

Table 2: What each custom exception means.

In main(), the whole body of the menu loop is wrapped in one try block with two catch clauses: catch (const LibraryException& e) catches anything from Table 2 and prints e.what(), and a final catch (const exception& e) acts as a safety net for anything unexpected (such as a memory allocation failure). Because the catch sits around the whole switch statement, a single bad menu choice or a missing book never crashes the program, the loop just prints the error and shows the menu again.

7. Program Flow

main() creates one Library object (this loads any saved data), then repeats a loop until the user chooses option 0: show the menu, read a choice with getValidatedChoice(), and run the matching case in a switch statement. Each case gathers whatever input it needs with getLine(), calls the matching Library method, and prints a confirmation. getValidatedChoice() specifically guards against non-numeric input: if cin fails to read an integer,

it clears the stream's error flags, discards the bad input left in the buffer, and throws `InvalidInputException` instead of leaving `cin` stuck in a broken state.

Final Output of Project:

```
+-----+
|           WELCOME           |
|       Library Management System       |
+-----+

Press any key to return to the main menu...|
```

Startup of Program

```
+-----+
|           LIBRARY MANAGEMENT SYSTEM           |
+-----+
| 1. Add Book                                     |
| 2. Add Magazine                               |
| 3. Remove Item                               |
| 4. Issue Item to Member                       |
| 5. Return Item                               |
| 6. Search Item (by title/ID)                  |
| 7. Display All Items                         |
| 8. Register New Member                       |
| 9. Display All Members                       |
| 10. View System Statistics                   |
| 11. Save Data Now                           |
| 0. Save & Exit                               |
+-----+

Enter your choice: |
```

Main Menu of Program

```
----- All Library Items (5) -----
[Book] ID: B001 | Title: The Great Gatsby | Author: F.Scot Fitzgerald | ISBN: 9780743273565 | Genre: Fiction | Status: Available
[Book] ID: B002 | Title: Clean Code | Author: Robert Martin | ISBN: 9780132350884 | Genre: Technology | Status: Available
[Book] ID: B003 | Title: Atomic Habits | Author: James Clear | ISBN: 9780735211292 | Genre: Self-Help | Status: Available
[Magazine] ID: M001 | Title: National Geographic | Publisher: National Geographic Society | Issue #: 245 | Status: Available
[Magazine] ID: M002 | Title: Time Magazine | Publisher: Time USA LLC | Issue #: 89 | Status: Available

Press any key to return to the main menu...|
```

Stored Some of Data to Show as Output Stored

8. Conclusion

The Library Management System satisfies every minimum criterion from the project brief inside a single C++ file: it models the domain with classes that use inheritance, polymorphism and abstraction; it keeps its data encapsulated behind the Library controller; it demonstrates a friend function, static members and const-correct methods; it persists all data to disk with fstream-based file handling that recovers gracefully from missing or corrupted files; and it validates every operation with a custom exception hierarchy caught through try/catch in the main menu loop.